



University of Idaho

Department of Computer Science

CS 4727/5727
Adversarial
Machine Learning

Dr. Alex Vakanski



Lecture 5

Black-box Evasion Attacks



Lecture Content

5.1 Evasion Attacks against Black-box Models

5.2 Gradient Estimation Attack: Bhagoji et al. (2017) Exploring the Space of Black-box Attacks on Deep Neural Networks

Student presentation by Asif Zaman

5.3 Boundary Attack: Brendel et al. (2018) Decision-Based Adversarial Attacks: Reliable Attacks Against Black-Box Machine Learning Models

5.4 Other Black-box Evasion Attacks

5.4.1 HopSkipJump Attack

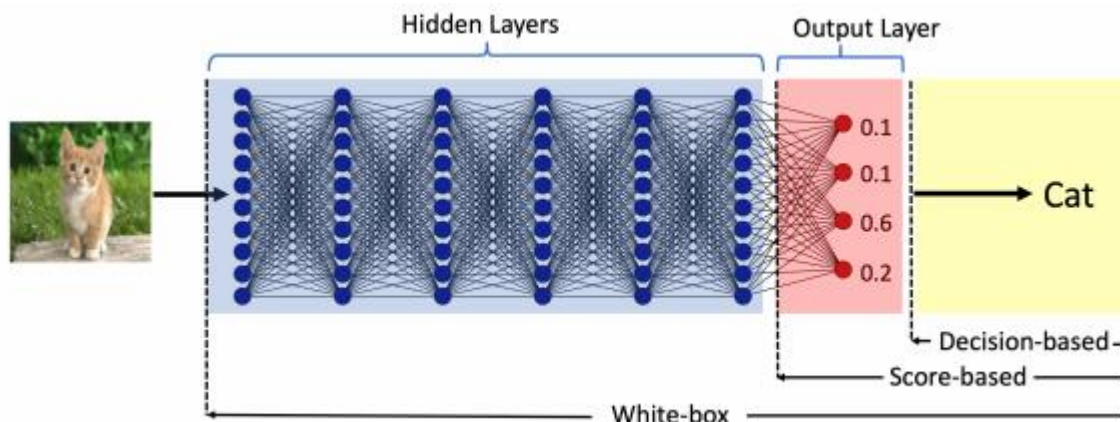
5.4.2 ZOO Attack

5.4.3 Simple Black-box Attack

Evasion Attacks against Black-box Models

5.1 Evasion Attacks against Black-box Models

- Black-box adversarial attacks can be classified into two categories:
 - *Query-based attacks*
 - The adversary queries the model and creates adversarial examples by using the provided information to queries
 - The queried model can provide:
 - Output class probabilities (i.e., confidence scores per class) used with **score-based attacks**
 - Output class, used with **decision-based attacks**
 - *Transfer-based attacks* (or *transferability attacks*)
 - The adversary does not query the model
 - The adversary trains its own substitute/surrogate local model, and transfers the adversarial examples to the target model
 - This type of approaches are also referred to as **zero queries attacks**



Gradient Estimation Attack

5.2 Gradient Estimation Attack

- *Gradient Estimation attack*
 - [Bhagoji, He, Li, Song \(2017\) Exploring the Space of Black-box Attacks on Deep Neural Networks](#)
- **Score-based** black-box attack
 - Based on query access to the model's class probabilities
 - Both targeted and untargeted attacks are achieved
- Validated on MNIST and CIFAR-10 datasets
 - The attack is also evaluated on real-world models hosted by Clarifai
- Advantages:
 - Outperformed other black-box attacks
 - Performance results are comparable to white-box attacks
 - Good results against adversarial defenses

Gradient Estimation Attack

5.2 Gradient Estimation Attack

- Gradient Estimation (GE) approach
 - Uses queries to directly estimate the gradient and carry out black-box attacks
 - The output to a query is the vector of class probabilities $\mathbf{p}^f(\mathbf{x})$ (i.e., confidence scores per class) for an input $\mathbf{x}$
 - The logits can also be recovered from the probabilities, by taking $\log(\mathbf{p}^f(\mathbf{x}))$
- The authors employed the **method of finite differences** for gradient estimation
 - Let $g(\mathbf{x})$ is a function whose gradient needs to be estimated
 - Finite difference (FD) estimation of the gradient of g with respect to input $\mathbf{x}$ is given by

$$\text{FD}_{\mathbf{x}}(g(\mathbf{x}), \delta) = \begin{bmatrix} \frac{g(\mathbf{x} + \delta \mathbf{e}_1) - g(\mathbf{x} - \delta \mathbf{e}_1)}{2\delta} \\ \vdots \\ \frac{g(\mathbf{x} + \delta \mathbf{e}_d) - g(\mathbf{x} - \delta \mathbf{e}_d)}{2\delta} \end{bmatrix}$$

- δ is a parameter that controls the estimation accuracy (selected 0.01 or 1)
- $\mathbf{e}_i$ are basis vectors such that $\mathbf{e}_i$ is 1 only for the i^{th} component and 0 everywhere else
- If the gradient exists, then the finite differences method can calculate an approximation of the gradient: $\lim_{\delta \rightarrow 0} \text{FD}_{\mathbf{x}}(g(\mathbf{x}), \delta) \approx \nabla_{\mathbf{x}} g(\mathbf{x})$

Gradient Estimation Attack

5.2 Gradient Estimation Attack

- **Approximate FGSM attack** with finite difference GE method
 - Gradient of a model f is taken with respect to the cross-entropy loss $\mathcal{L}_f(\mathbf{x}, \mathbf{y})$
 - For input $\mathbf{x}$ with true class label y , the loss is

$$\mathcal{L}_f(\mathbf{x}, \mathbf{y}) = - \sum_{j=1}^{|\mathbf{y}|} y_j \log p_j^f(\mathbf{x}) = -\log(p_y^f(\mathbf{x}))$$

- Recall that the derivative of a log function is $\frac{d}{dx} \log(x) = \frac{1}{x}$ and thus $\frac{d}{dx} \log(h(x)) = \frac{h'(x)}{h(x)}$
- Therefore, the gradient of the loss function $\ell_f(\mathbf{x}, y)$ with respect to the input $\mathbf{x}$ is

$$\nabla_{\mathbf{x}} \mathcal{L}_f(\mathbf{x}, \mathbf{y}) = \nabla_{\mathbf{x}} \left(-\log(p_y^f(\mathbf{x})) \right) = - \frac{\nabla_{\mathbf{x}} p_y^f(\mathbf{x})}{p_y^f(\mathbf{x})}$$

- An untargeted FGSM adversarial sample can be generated by using the FD estimate of the gradient $\nabla_{\mathbf{x}} p_y^f(\mathbf{x})$, i.e.,

$$\mathbf{x}_{\text{adv}} = \mathbf{x} + \epsilon \cdot \text{sign} \left(\frac{\text{FD}_{\mathbf{x}}(p_y^f(\mathbf{x}), \delta)}{p_y^f(\mathbf{x})} \right)$$

- Similarly, a targeted FGSM adversarial sample with class T can be found by using

$$\mathbf{x}_{\text{adv}} = \mathbf{x} - \epsilon \cdot \text{sign} \left(\frac{\text{FD}_{\mathbf{x}}(p_T^f(\mathbf{x}), \delta)}{p_T^f(\mathbf{x})} \right)$$

Gradient Estimation Attack

5.2 Gradient Estimation Attack

- **Approximate C-W attack** with finite difference GE method
 - Carlini & Wagner attack uses a loss function based on the logits values $\phi(\cdot)$

$$\mathcal{L}(\mathbf{x}, y) = \max(\phi(x + \delta)_y - \max\{\phi(x + \delta)_i : i \neq y\}, -\kappa)$$

- Logits values $\phi(\cdot)$ can be computed by taking the logarithm of the softmax probabilities, up to an additive constant
- For an **untargeted C-W attack**, the loss is the difference between the logits for the true class y and the second-most-likely class y' , i.e., $\phi(x + \delta)_y - \phi(x + \delta)_{y'}$
 - Since the loss is the difference of logits, the additive constant is canceled
 - By using FD approximation of the gradient, it is obtained

$$\mathbf{x}_{\text{adv}} = \mathbf{x} + \epsilon \cdot \text{sign}(\text{FD}_{\mathbf{x}}(\phi(\mathbf{x})_{y'} - \phi(\mathbf{x})_y, \delta))$$

- For a **targeted C-W attack**, the adversarial sample is

$$\mathbf{x}_{\text{adv}} = \mathbf{x} - \epsilon \cdot \text{sign}(\text{FD}_{\mathbf{x}}(\max(\phi(\mathbf{x})_i : i \neq T) - \phi(\mathbf{x})_T, \delta))$$

Gradient Estimation Attack

5.2 Gradient Estimation Attack

- **Iterative FGSM attack** with finite difference GE method
 - This is similar to the Projected Gradient Descent attack, which uses several iterations of the FGSM attack and achieves higher success rate than the single step FGSM attack
 - An iterative FD attack with $t + 1$ iterations using the cross-entropy loss is

$$\mathbf{x}_{\text{adv}}^{t+1} = \mathbf{x}_{\text{adv}}^t + \alpha \cdot \text{sign} \left(\frac{\text{FD} \left(\nabla_{\mathbf{x}_{\text{adv}}^t} p_y^f(\mathbf{x}_{\text{adv}}^t), \delta \right)}{p_y^f(\mathbf{x}_{\text{adv}}^t)} \right)$$

- **Iterative C-W attack** is also applied in a similar manner by modifying the single-step approach presented on the previous page

$$\mathbf{x}_{\text{adv}}^{t+1} = \mathbf{x}_{\text{adv}}^t + \alpha \cdot \text{sign} \left(\text{sign} \left(\text{FD}(\phi(x)_{y'}, -\phi(x)_y, \delta) \right) \right)$$

Experimental Validation

5.2 Gradient Estimation Attack

- Validation of **non-targeted black-box attacks** using Gradient Estimation with FD
 - The table presents the success rate and average distortion (in parenthesis)
 - Baseline methods:
 - D. of M. – Difference of Means attack, uses the mean difference between the true class and the target class as added perturbation
 - Rand. – Random perturbation by adding random noise from a distribution (e.g., Gaussian)
 - ‘xent’ is for cross-entropy loss, ‘logit’ is C-W logits loss, ‘I’ is iterative
 - MNIST with L_∞ constraint of $\epsilon = 0.3$, and CIFAR-10 with L_∞ constraint of $\epsilon = 8$
 - Iterative C-W attack (IFD-logit) produced best results

MNIST	Baseline		Gradient Estimation using Finite Differences			
Model	D. of M.	Rand.	Single-step		Iterative	
			FD-xent	FD-logit	IFD-xent	IFD-logit
A	44.8 (5.6)	8.5 (6.1)	51.6 (3.3)	92.9 (6.1)	75.0 (3.6)	100.0 (2.1)
B	81.5 (5.6)	7.8 (6.1)	69.2 (4.5)	98.9 (6.3)	86.7 (3.9)	100.0 (1.6)
C	20.2 (5.6)	4.1 (6.1)	60.5 (3.8)	86.1 (6.2)	80.2 (4.5)	100.0 (2.2)
D	97.1 (5.6)	38.5 (6.1)	95.4 (5.8)	100.0 (6.1)	98.4 (5.4)	100.0 (1.2)
CIFAR-10	Baseline		Gradient Estimation using Finite Differences			
Model	D. of M.	Rand.	Single-step		Iterative	
			FD-xent	FD-logit	IFD-xent	IFD-logit
Resnet-32	9.3 (440.5)	19.4 (439.4)	49.1 (217.1)	86.0 (410.3)	62.0 (149.9)	100.0 (65.7)
Resnet-28-10	6.7 (440.5)	17.1 (439.4)	50.1 (214.8)	88.2 (421.6)	46.0 (120.4)	100.0 (74.9)
Std.-CNN	20.3 (440.5)	22.2 (439.4)	80.0 (341.3)	98.9 (360.9)	66.0 (202.5)	100.0 (79.9)

Experimental Validation

5.2 Gradient Estimation Attack

- Validation of **targeted black-box attacks** using Gradient Estimation with FD
 - Iterative FGSM (IFD-xent) attack produced best results on MNIST
 - Iterative C-W (IFD-logit) attack produced best results on CIFAR-10

MNIST	Baseline	Gradient Estimation using Finite Differences			
Model	D. of M.	Single-step		Iterative	
		FD-xent	FD-logit	IFD-xent	IFD-logit
A	15.0 (5.6)	30.0 (6.0)	29.9 (6.1)	100.0 (4.2)	99.7 (2.7)
B	35.5 (5.6)	29.5 (6.3)	29.3 (6.3)	99.9 (4.1)	98.7 (2.4)
C	5.84 (5.6)	34.1 (6.1)	33.8 (6.4)	100.0 (4.3)	99.8 (3.0)
D	59.8 (5.6)	61.4 (6.3)	60.8 (6.3)	100.0 (3.7)	99.9 (1.9)
CIFAR-10	Baseline	Gradient Estimation using Finite Differences			
Model	D. of M.	Single-step		Iterative	
		FD-xent	FD-logit	IFD-xent	IFD-logit
Resnet-32	1.2 (440.3)	23.8 (439.5)	23.0 (437.0)	100.0 (110.9)	100.0 (89.5)
Resnet-28-10	0.9 (440.3)	29.2 (439.4)	28.0 (436.1)	100.0 (123.2)	100.0 (98.3)
Std.-CNN	2.6 (440.3)	44.5 (439.5)	40.3 (434.9)	99.0 (178.8)	95.0 (126.8)



Query Reduction

5.2 Gradient Estimation Attack

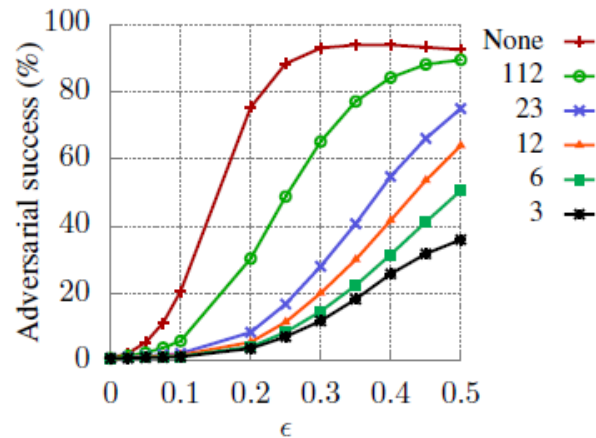
- Shortcoming of the proposed approach:
 - Requires $O(d)$ queries per input, where d is the dimension of the input (e.g., number of pixels in images)
 - The presented FD approximation required $2 \cdot d$ queries
 - E.g., for FGSM attack on 28×28 pixels = 784 pixels it requires $2 \cdot 784 = 1,568$ queries
- The authors propose two approaches for reducing the number of queries
 - Random grouping
 - The gradient is estimated only for a random group of selected pixels, instead of estimating the gradient per each pixel
 - PCA (Principal Component Analysis)
 - Compute the gradient only along a number of principal component vectors

Query Reduction

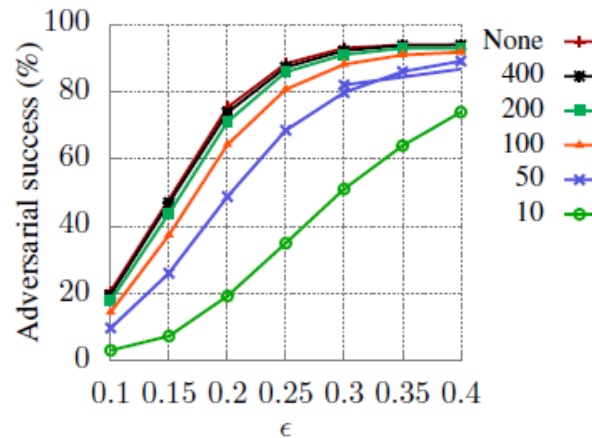
5.2 Gradient Estimation Attack

- Validation of the methods for query reduction
 - For random grouping, the success rate decreases with decreasing the group size (left figure)
 - I.e., using only 3 group of pixels to estimate the gradient is less efficient than using 112 groups of pixels
 - For PCA, the success rate decreases as the number of PC is decreased (middle and right figure)
 - The success rate is still high for smaller number of PC

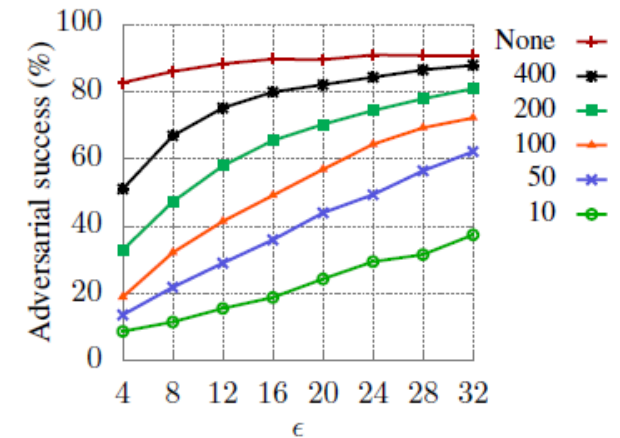
Random feature groupings for Model A



PCA-based query reduction for Model A



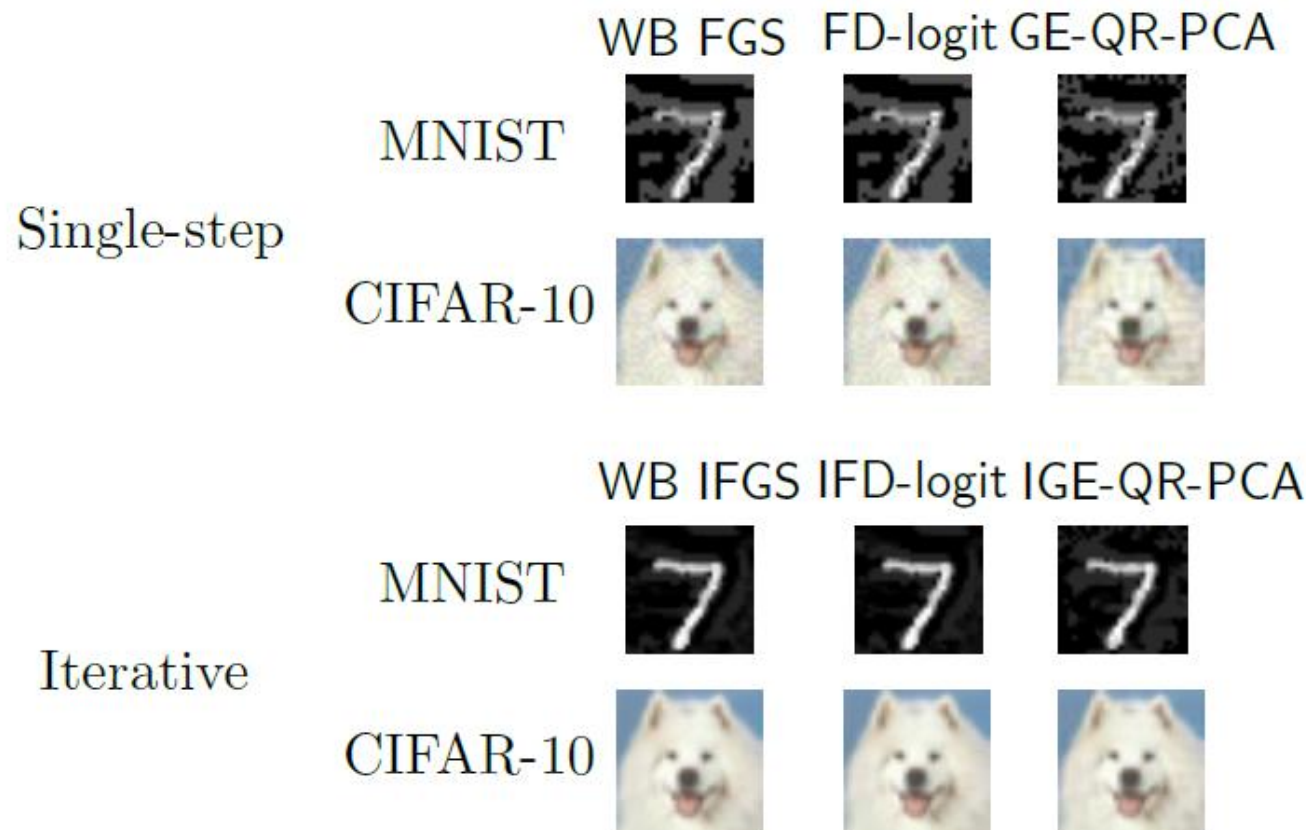
PCA-based query reduction for Resnet-32



Adversarial Samples

5.2 Gradient Estimation Attack

- Non-targeted adversarial samples
 - WB-IFGS – white-box iterative FGSM attack
 - IFD-logit – black-box iterative C&W attack (logit loss)
 - IGE-QR-PCA - black-box Iterative Gradient Estimation with Query Reduction using PCA



Attacks on Real Models

5.2 Gradient Estimation Attack

- Attacks on two real-world models hosted by Clarifai
 - Not Safe For Work (NSFW) model
 - Two categories: 'safe', 'not safe'
 - Content Moderation model
 - Five categories: 'safe', 'suggestive', 'explicit', 'drug,' and 'gore'
 - Example: an adversary could upload violent adversarially-modified images, which may be marked incorrectly as 'safe' by the Content Moderation model



Original image
Class: 'drug'
Confidence: 0.99



Adversarial image
Class: 'safe'
Confidence: 0.96

Decision-Based Adversarial Attacks: Reliable Attacks Against Black-Box Machine Learning Models

ICLR 2018, Wieland Brendel, Jonas

Presented By: Asif Zaman Course:
Adversarial Machine Learning

Motivation & The Real-World Problem

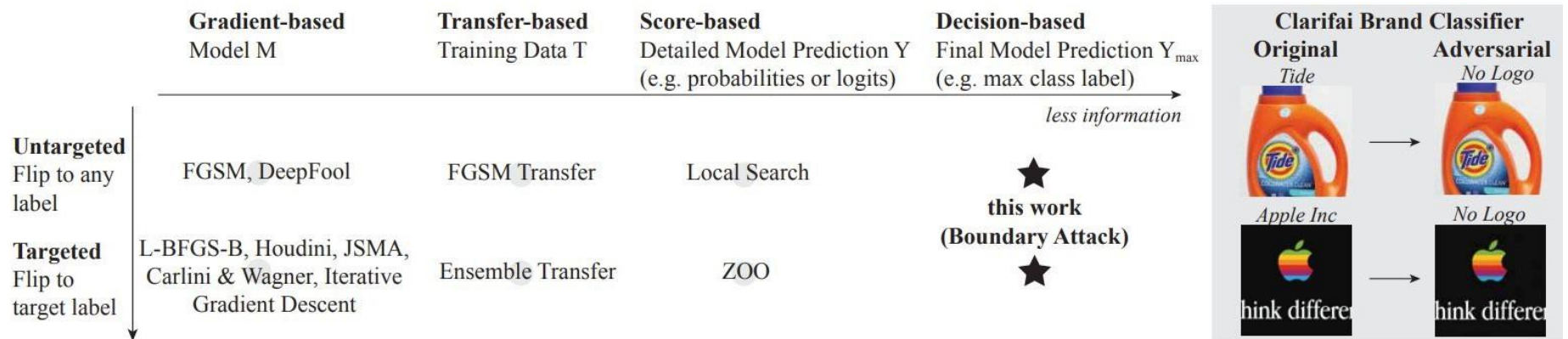
Adversarial Examples are Ubiquitous: Imperceptible perturbations can drastically alter model predictions (e.g., VGG-19: "tiger cat" -> "bus").

The Threat Model Gap: Most academic attack research relies on information rarely available in real-world scenarios

Real-World Scenario: Deployed systems like autonomous cars, face recognition security, or cloud APIs often act as pure "Black Boxes."

The only observable output is the final decision (e.g., class label).

Taxonomy of Adversarial Attacks



Gradient-based (White-box): Needs detailed model info (weights, gradients). Easily defended via gradient masking. (e.g., FGSM, C&W).

Transfer-based: Needs training data to build a substitute model. Can be defended against via adversarial training on ensembles.

Score-based: Uses predicted probabilities/logits to estimate gradients. Can be impeded by stochasticity or saturated probabilities. (e.g., ZOO).

Decision-based: Only relies on the final output label $Y_{\max}$.

The Challenge of Decision-Based Attacks

The Information Constraint: How do you optimize a perturbation to minimize distance when you only get a binary signal? You cannot compute a gradient.

Prior State-of-the-Art decision-based attacks were limited to

- simple linear/convex models
- simple datasets (e.g., MNIST).

Goal of this Paper: Introduce the first effective decision-based attack applicable to DNN on natural image datasets (ImageNet, CIFAR).

Introducing The Boundary Attack

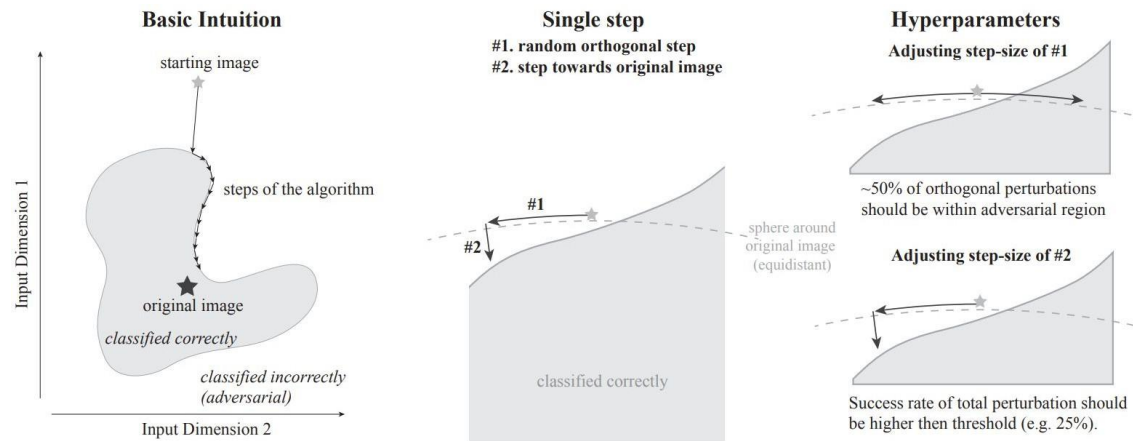
A Paradigm Shift in Attack Logic:

- Traditional Attacks (e.g., FGSM, C&W): Start with the original image and iteratively add small perturbations until it crosses the decision boundary.
- The Boundary Attack: Starts with a large perturbation that is already adversarial, and iteratively reduces the perturbation while staying on the adversarial side of the boundary.

Mechanism: It is essentially a rejection sampling algorithm that performs a random walk along the decision boundary.

Features: Conceptually simple, requires little hyperparameter tuning, and does not need substitute models.

Boundary Attack Intuition Visualized



Initialization: Start at a point $\tilde{o}_0$ that is already adversarial (e.g., random uniform noise for untargeted attack).

The Goal: Move from the starting adversarial point toward the original image, while staying in the adversarial region.

The Steps:

- Move along a sphere around the original image to explore the boundary (Orthogonal perturbation).
- Take a small step inwards towards the original image (reducing distance).

The Algorithm (Simplified)

1. Initialization: $k=0$, $\tilde{o}_0 \sim U(0,1)$ such that $\tilde{o}_0$ is adversarial.
2. Loop while $k < \text{max_steps}$:
3. Draw a random perturbation η_k from a proposal distribution $P(\tilde{o}_{k-1})$.
4. Check constraints: Is the new point $\tilde{o}_{k-1} + \eta_k$ still adversarial?
 - a. If Yes (Success): Update position: $\tilde{o}_k = \tilde{o}_{k-1} + \eta_k$. This new point is closer to the original image.
 - b. If No (Reject): Stay put: $\tilde{o}_k = \tilde{o}_{k-1}$.

Note: The efficiency depends crucially on the proposal distribution P .

Proposal Distribution & Hyperparameters

The proposal distribution attempts to satisfy three constraints simultaneously:

- Remain within input domain (e.g., $[0, 255]$ pixel values).
- Maintain a relative step size δ (exploring the boundary).
- Reduce distance to the original input by relative amount ε (moving inward).

Dynamic Adjustment (Trust Region style):

Step sizes δ and ε are adjusted dynamically based on success rate of recent steps to adapt to the local geometry of the decision boundary.

Experimental Setup

Datasets: MNIST, CIFAR-10, ImageNet-1000.

Models: Standard CNNs for MNIST/CIFAR; VGG-19, ResNet-50, Inception-v3 for ImageNet.

Baselines (Gradient-based): FGSM, DeepFool, Carlini & Wagner (C&W).

Metric: Median squared L2-distance of the adversarial perturbation across samples. $S_A(M) = \text{median}_i (1/N \sum_j ||\eta(o_i)||_2^2)$

Task: primarily focusing on Untargeted attacks (flip to any wrong label), though targeted is possible.

Results: Comparison with White-Box Attacks

	Attack Type	MNIST	CIFAR	VGG-19	ResNet-50	Inception-v3
FGSM	gradient-based	4.2e-02	2.5e-05	1.0e-06	1.0e-06	9.7e-07
DeepFool	gradient-based	4.3e-03	5.8e-06	1.9e-07	7.5e-08	5.2e-08
Carlini & Wagner	gradient-based	2.2e-03	7.5e-06	5.7e-07	2.2e-07	7.6e-08
Boundary	decision-based	3.6e-03	5.6e-06	2.9e-07	1.0e-07	6.5e-08

The Cost of Minimal Information (Efficiency)

The Boundary Attack achieves white-box quality results, but at a high computational cost (number of queries).

	forward passes	backward passes
DeepFool	~7	~37
C&W	~16,000	~16,000
Boundary Attack	1,200,000	0

Takeaway: If you need imperceptible attacks on a strict black box, it is possible, but expensive.

Robustness: Breaking Defensive Distillation

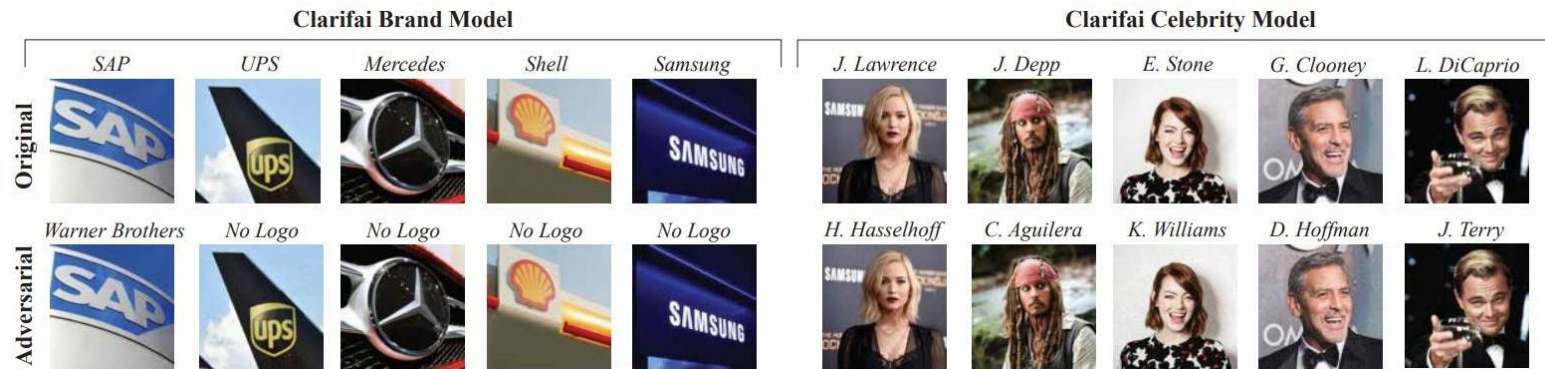
Defensive Distillation (Papernot et al., 2016): A prominent defense that trains a network on softmax outputs using high temperature, reducing gradient magnitudes at test time.

The Issue: It was later shown to be a form of "gradient masking." It stops gradient-based attacks like FGSM, but fails against attacks that work on logits (like C&W).

Boundary Attack Performance:

- Because the Boundary Attack does not use gradients or probabilities, it is totally immune to this defense.
- **Results (CIFAR Distilled):** FGSM fails entirely. Boundary attack succeeds with perturbation sizes comparable to undiscilled networks (1.3×10^{-5} vs 5.6×10^{-6}).

Real-World Application: Clarifai.com



True Black Box Test: Applied to commercial cloud-based computer vision APIs (Clarifai brand detector and celebrity recognizer).

Constraint: The attack only received the name of the identified object (no confidence scores).

Results: Successfully generated adversarial examples.

- Some perturbations were imperceptible (like ImageNet results).
- Others required larger, slightly noticeable noise (order of 10^{-2} or 10^{-3} rather than 10^{-7}).

Discussion & Strengths

Why is this paper important?

- **Realistic Threat Model:** Addresses the scenario most relevant to deployed ML systems where internals are hidden.
- **Simplicity:** The core algorithm is a straightforward rejection sampling method, yet effective.
- **Flexibility:** Can use almost any adversarial criterion (misclassification, targeted, top-k) as long as an initial adversarial point can be found.
- **Defeating Defenses:** Highlights that defences relying on masking gradients or scores are ineffective against decision-based agents.

Conclusion & Takeaways

Decision-based attacks are a critical, distinct category of adversarial threats.

The **Boundary Attack** was the first to scale to complex datasets like ImageNet using only the model's final decision.

It inverts standard attack logic: start adversarial, reduce noise.

It achieves perturbation magnitudes competitive with state-of-the-art white-box attacks, though at a much higher query cost.

Major Implication for Robustness Research: A model cannot be considered robust just because gradient-based or score-based attacks fail. Decision-based attacks must be part of the evaluation suite.



HopSkipJump Attack

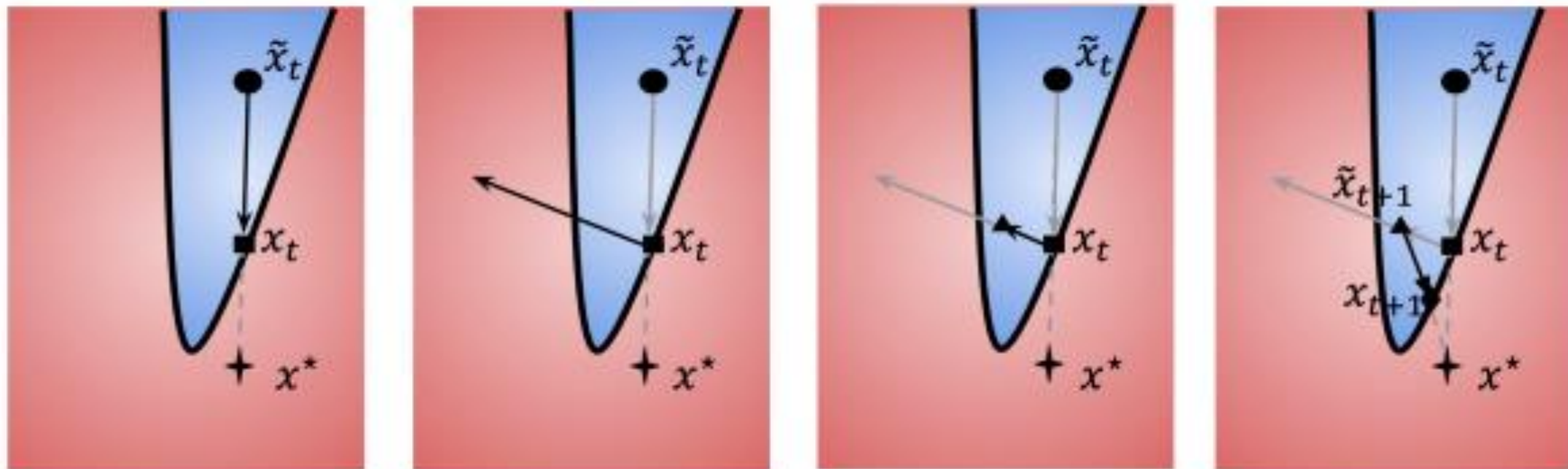
5.4.1 HopSkipJump Attack

- *HopSkipJump Attack*
 - [Chen and Jordan \(2019\) HopSkipJumpAttack: A Query-efficient Decision-based Adversarial Attack](#)
- This attack is an extension of the Boundary Attack
 - I.e., it is a **decision-based attack**, and therefore has access only to the predicted output class
 - HopSkipJump Attack requires significantly **fewer queries** than the Boundary Attack
 - It includes both untargeted and targeted attacks
 - Proposes a novel approach for estimation of the gradient direction along the decision boundary

HopSkipJump Attack

5.4.1 HopSkipJump Attack

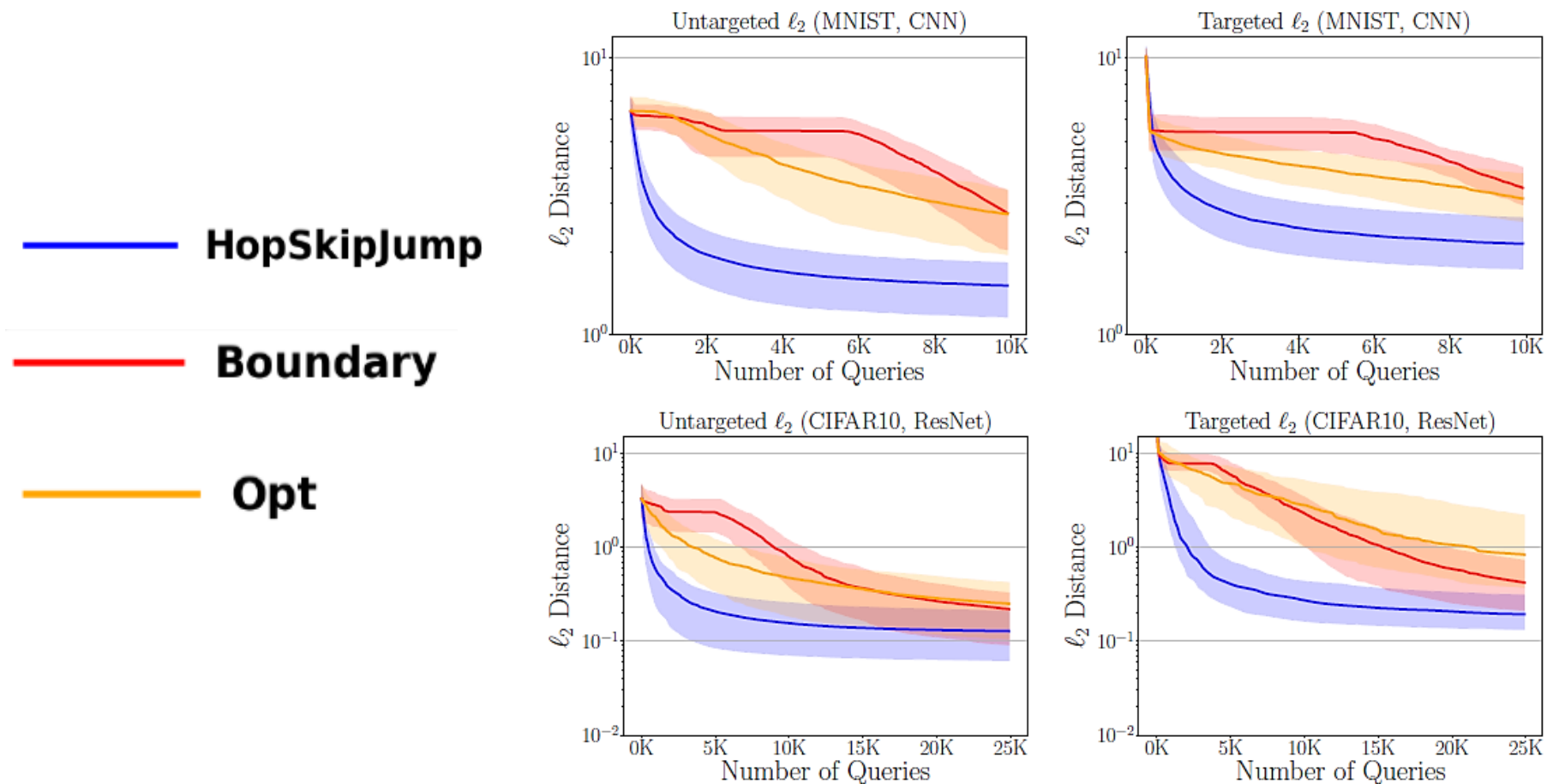
- Approach:
 1. Start from an adversarial image $\tilde{x}_t$
 2. Perform a binary search to the original image x^* to find the boundary (left figure)
 3. Estimate the gradient direction at the boundary point x_t (second figure from left)
 4. Perform a step-size move in adversarial direction, and update to the next image $\tilde{x}_{t+1}$ (second figure from right)
 5. Search again for the next boundary point x_{t+1} (right figure)
 6. Repeat until the closest adversarial image to the original image x^* is found



HopSkipJump Attack

5.4.1 HopSkipJump Attack

- Experimental evaluation
 - Comparison to Boundary attack and Opt attack on CIFAR-10
 - HopSkipJump (blue curve) achieves lower ℓ_2 perturbation using fewer queries



HopSkipJump Attack

5.4.1 HopSkipJump Attack

- Untargeted attack
 - 2nd to 9th columns: images at 100, 200, 500, 1K, 2K, 5K, 10K, 25K queries
 - The original image for the attack is shown on the right



- Targeted attack



ZOO Attack

5.4.2 ZOO Attack

- *ZOO attack*
 - [Chen \(2017\) Zoo: Zeroth-order optimization based black-box attacks to deep neural networks without training substitute models](#)
- **Zeroth-order optimization** refers to optimization based on access to the function values $f(x)$ only
 - As opposed to first-order optimization via the gradient $\nabla f(x)$
 - E.g., score-based and decision-based black-box approaches are zeroth-order optimization methods, as they don't require the gradient information
- ZOO attack has similarities with the Gradient Estimation Attack
- It is a **score-based** black-box version of the Carlini-Wagner attack

Adversarial Attack

5.4.2 ZOO Attack

- Recall again that the **Gradient Estimation attack** uses the **finite difference** approach to approximate the gradient as $\mathbf{g} = \nabla_{\mathbf{x}} f(\mathbf{x}) \approx \frac{f(\mathbf{x}+h) - f(\mathbf{x}-h)}{2h}$
 - E.g., if the intensity of a pixel x_i is 150, and $h = 10$, then we will query the model to give us the predictions for $f(150 + 10) = f(160)$ and for $f(150 - 10) = f(140)$, so we can estimate the gradient $\widehat{\mathbf{g}}_i = \nabla_{x_i} f(\mathbf{x})$ for the pixel x_i
 - We need to do 2 queries for each pixel, and for an images with 28×28 pixels = 784 pixels, we need to do $2 \cdot 784 = 1,568$ queries to estimate the gradient
- ZOO attack** solves an optimization, similar to C&W targeted white-box attack

$$\begin{aligned} &\text{minimize } \|\mathbf{x} - \mathbf{x}_0\|_2^2 + c \cdot (Z(\mathbf{x})_{y'} - Z(\mathbf{x})_T) \\ &\text{subject to } \mathbf{x} \in [0,1] \end{aligned}$$

- ZOO solves the optimization problem with the FD estimated loss based on:

$$\begin{aligned} &\text{minimize } \|\mathbf{x} - \mathbf{x}_0\|_2^2 + c \cdot FD(Z(\mathbf{x})_{y'} - Z(\mathbf{x})_T, h) \\ &\text{subject to } \mathbf{x} \in [0,1] \end{aligned}$$

- Adam optimization** is used to solve the problem

Adam Optimization Attack

5.4.2 ZOO Attack

- Algorithm for the ZOO attack using Adam optimization

Algorithm 2 ZOO-ADAM: Zeroth Order Stochastic Coordinate Descent with Coordinate-wise ADAM

Require: Step size η , ADAM states $M \in \mathbb{R}^p, v \in \mathbb{R}^p, T \in \mathbb{Z}^p$, ADAM hyper-parameters $\beta_1 = 0.9, \beta_2 = 0.999, \epsilon = 10^{-8}$

- 1: $M \leftarrow 0, v \leftarrow 0, T \leftarrow 0$
 - 2: **while** not converged **do**
 - 3: Randomly pick a coordinate $i \in \{1, \dots, p\}$
 - 4: Estimate $\hat{g}_i$ using (6)
 - 5: $T_i \leftarrow T_i + 1$
 - 6: $M_i \leftarrow \beta_1 M_i + (1 - \beta_1) \hat{g}_i, \quad v_i \leftarrow \beta_2 v_i + (1 - \beta_2) \hat{g}_i^2$
 - 7: $\hat{M}_i = M_i / (1 - \beta_1^{T_i}), \quad \hat{v}_i = v_i / (1 - \beta_2^{T_i})$
 - 8: $\delta^* = -\eta \frac{\hat{M}_i}{\sqrt{\hat{v}_i + \epsilon}}$
 - 9: Update $x_i \leftarrow x_i + \delta^*$
 - 10: **end while**
-

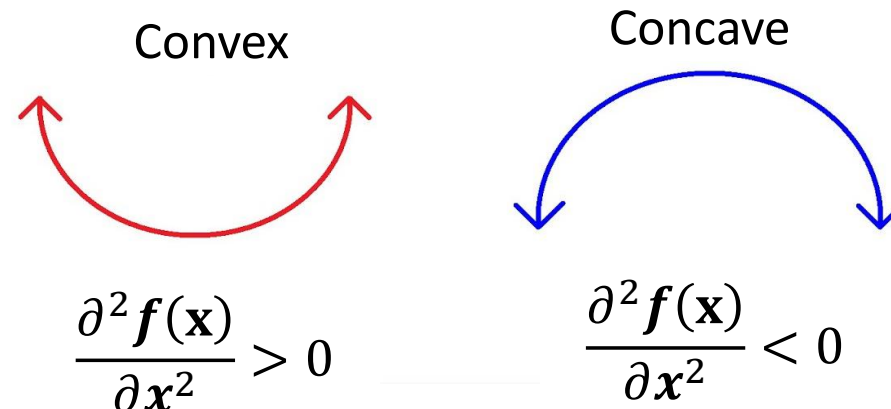
Newton Optimization Attack

5.4.2 ZOO Attack

- The paper proposed one more similar approach, that instead of Adam optimization uses *Newton optimization* method
 - Newton optimization method finds a minimum of $f(x)$ by performing the following iterations: $x_{k+1} = x_k - \frac{f'(x_k)}{f''(x_k)}$
- The approximation of the **Hessian matrix** of the model is estimated based on

$$\mathbf{h} = \frac{\partial^2}{\partial \mathbf{x}^2} f(\mathbf{x}) \approx \frac{f(\mathbf{x}+\mathbf{h}) - 2f(\mathbf{x}) + f(\mathbf{x}-\mathbf{h})}{\mathbf{h}^2}$$

- If $\mathbf{h} > \mathbf{0}$, then the loss function is convex, update is based on $\mathbf{g}/\mathbf{h}$ (i.e., $x_k - \frac{f'(x_k)}{f''(x_k)}$)
- If $\mathbf{h} \leq \mathbf{0}$, then the loss function is concave, update is based only on the gradient $\mathbf{g}$ (i.e., $x_k - f'(x_k)$)



Newton Optimization Attack

5.4.2 ZOO Attack

- Algorithm for the ZOO attack with Newton optimization

Algorithm 3 ZOO-Newton: Zeroth Order Stochastic Coordinate Descent with Coordinate-wise Newton's Method

Require: Step size η

```
1: while not converged do
2:   Randomly pick a coordinate  $i \in \{1, \dots, p\}$ 
3:   Estimate  $\hat{g}_i$  and  $\hat{h}_i$  using (6) and (7)
4:   if  $\hat{h}_i \leq 0$  then
5:      $\delta^* \leftarrow -\eta \hat{g}_i$ 
6:   else
7:      $\delta^* \leftarrow -\eta \frac{\hat{g}_i}{\hat{h}_i}$ 
8:   end if
9:   Update  $\mathbf{x}_i \leftarrow \mathbf{x}_i + \delta^*$ 
10: end while
```

Experimental Evaluation

5.4.2 ZOO Attack

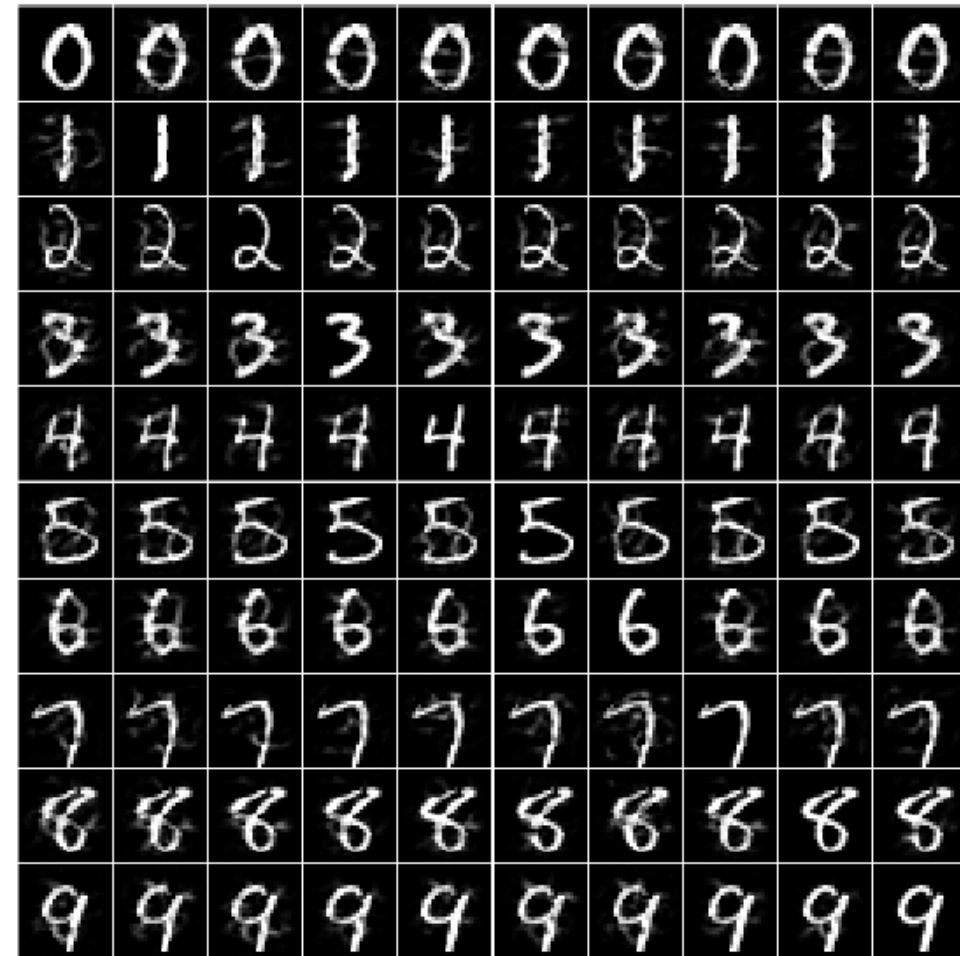
- On MNIST and Cifar-10, ZOO attacks achieved almost 100% success rate
 - The added L_2 perturbations are comparable to C&W white-box attack
 - As expected, the time for generating adversarial samples is longer than white-box attacks

	MNIST					
	Untargeted			Targeted		
	Success Rate	Avg. L_2	Avg. Time (per attack)	Success Rate	Avg. L_2	Avg. Time (per attack)
White-box (C&W)	100 %	1.48066	0.48 min	100 %	2.00661	0.53 min
Black-box (Substitute Model + FGSM)	40.6 %	-	0.002 sec (+ 6.16 min)	7.48 %	-	0.002 sec (+ 6.16 min)
Black-box (Substitute Model + C&W)	33.3 %	3.6111	0.76 min (+ 6.16 min)	26.74 %	5.272	0.80 min (+ 6.16 min)
Proposed black-box (ZOO-ADAM)	100 %	1.49550	1.38 min	98.9 %	1.987068	1.62 min
Proposed black-box (ZOO-Newton)	100 %	1.51502	2.75 min	98.9 %	2.057264	2.06 min
	CIFAR10					
	Untargeted			Targeted		
	Success Rate	Avg. L_2	Avg. Time (per attack)	Success Rate	Avg. L_2	Avg. Time (per attack)
White-box (C&W)	100 %	0.17980	0.20 min	100 %	0.37974	0.16 min
Black-box (Substitute Model + FGSM)	76.1 %	-	0.005 sec (+ 7.81 min)	11.48 %	-	0.005 sec (+ 7.81 min)
Black-box (Substitute Model + C&W)	25.3 %	2.9708	0.47 min (+ 7.81 min)	5.3 %	5.7439	0.49 min (+ 7.81 min)
Proposed Black-box (ZOO-ADAM)	100 %	0.19973	3.43 min	96.8 %	0.39879	3.95 min
Proposed Black-box (ZOO-Newton)	100 %	0.23554	4.41 min	97.0 %	0.54226	4.40 min

Experimental Evaluation

5.4.2 ZOO Attack

- Comparison between C&W white-box (left) and ZOO attack (right)



Queries Reduction

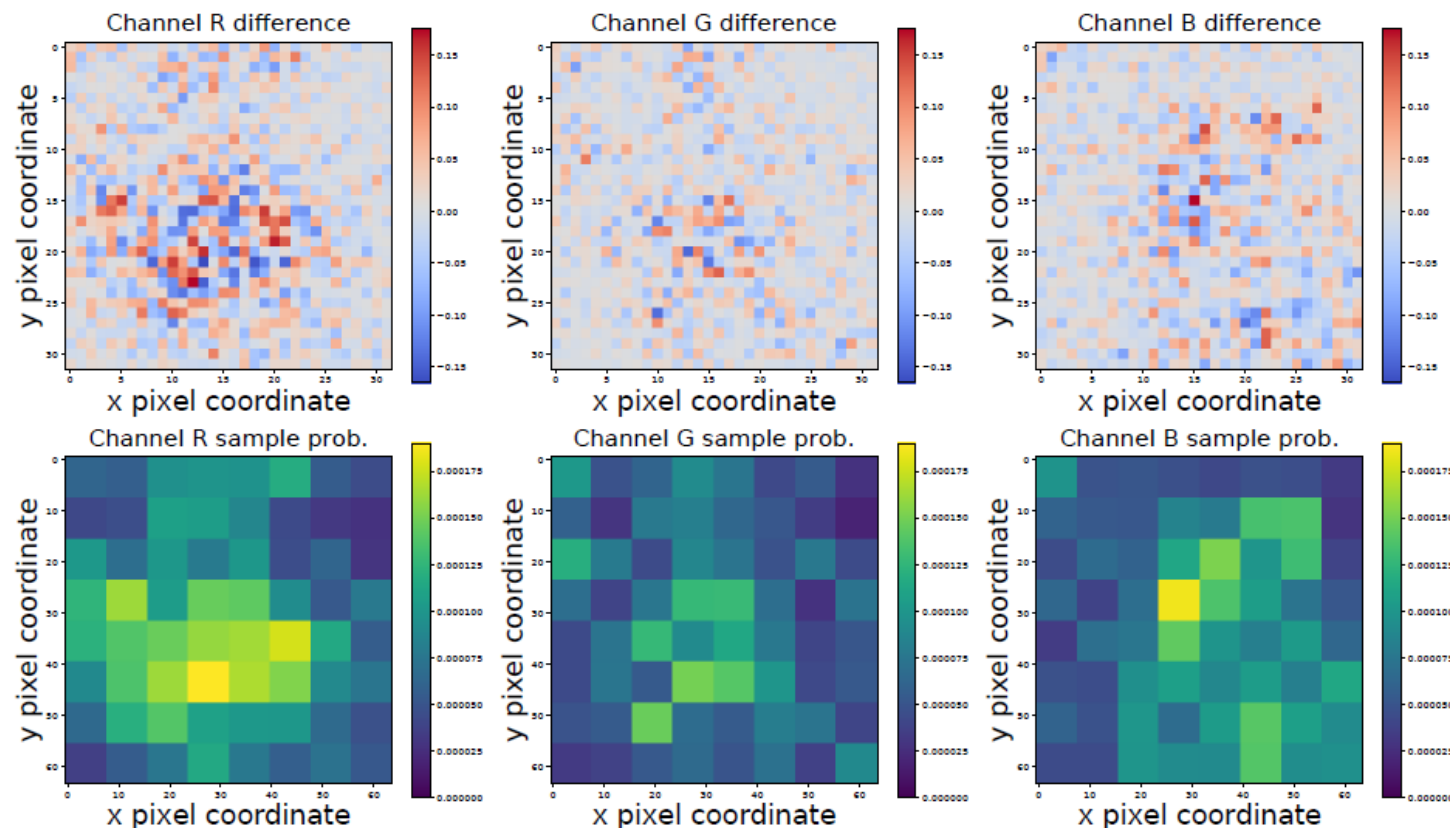
5.4.2 ZOO Attack

- The authors proposed techniques to reduce the number of queries
 - Note that for 28×28 pixels, we need $2 \cdot 784 = 1,568$ queries to estimate the gradient
 - Recall that PCA and random sets of pixels were used in Gradient Estimation attack
- The proposed approach starts with reduced resolution, and the resolution is progressively increased (referred to as **hierarchical attack**)
 - E.g., an original image of a size 299×299 pixels is used
 - Divide the image into 8×8 regions
 - Make only 64 queries to estimate the gradients
 - Optimize until the loss start decreasing
 - Increase to 16×16 regions
 - Make queries and optimize until the loss start decreasing
 - Increase to 32×32 regions
 - Make queries and optimize until the loss start decreasing
 - Repeat until the attack is successful

Queries Reduction

5.4.2 ZOO Attack

- Another technique for query reduction is based on *importance sampling*
 - Estimate the gradient only for the most important regions in an image
 - Upper figures show the gradient for the Red, Green, and Blue channels
 - » E.g., corner pixels are less important for this image, and the changes in R are more important than G and B channels
 - Lower figures shows the most important pixels for R, G, B channels, that are queried first



bagel



Experimental Evaluation

5.4.2 ZOO Attack

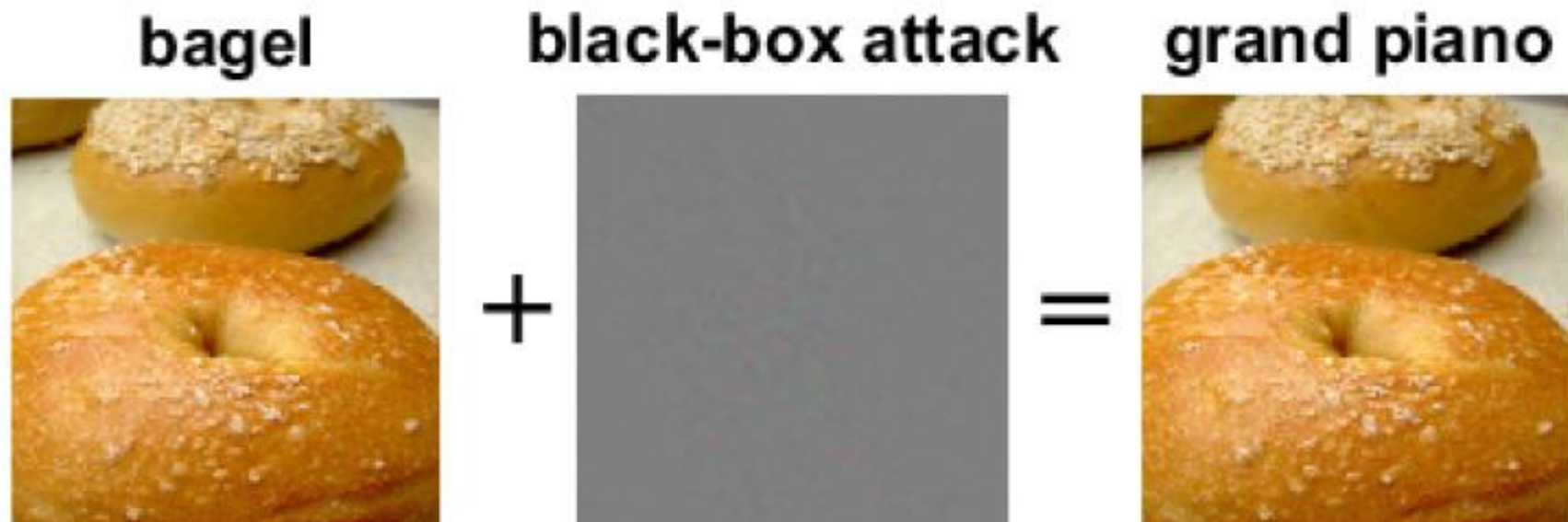
- ImageNet untargeted attack
 - Recall that there are 1,000 classes in ImageNet
 - InceptionV3 model used
 - ZOO attack required about 192,000 queries per image, 20 minutes per image
 - The success rate is lower than C&W white-box attack, but is still high

	Success Rate	Avg. L_2
White-box (C&W)	100 %	0.37310
Proposed black-box (ZOO-ADAM)	88.9 %	1.19916

Examples

5.4.2 ZOO Attack

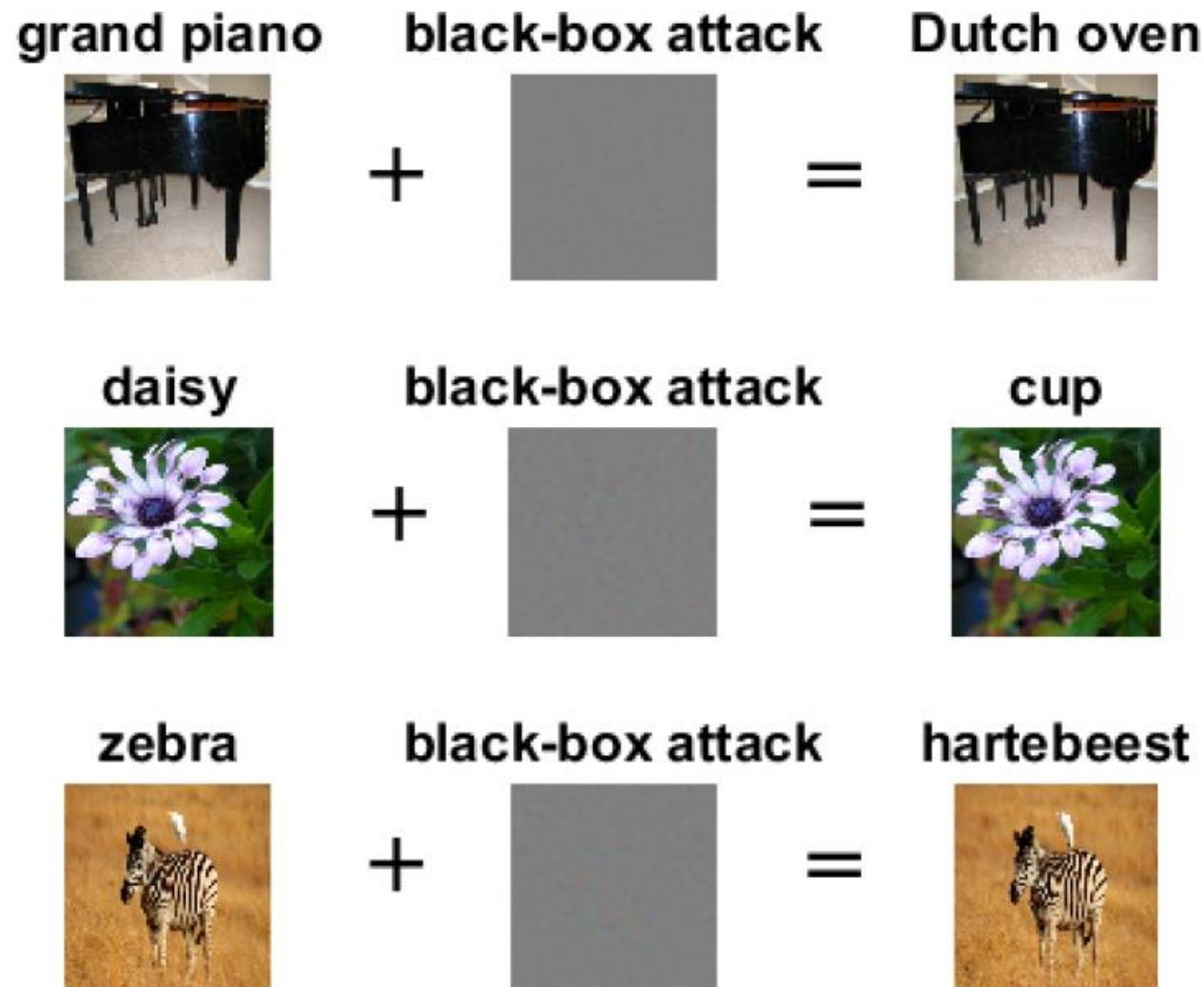
- Targeted attack
 - The added perturbations are imperceptible



Examples

5.4.2 ZOO Attack

- Untargeted attack



SimBA Attack

5.4.3 Simple Black-box Attack

- *Simple Black-box Attack*
 - [Guo et al. \(2019\) Simple Black-box Adversarial Attacks](#)
- A.k.a. SimBA attack
 - **Score-based attack** (using probability vectors)
 - Focus on query efficiency
 - Both targeted and untargeted attacks were demonstrated
- Approach:
 - Use random orthonormal perturbations for each query
 - Focus on regions in images with high-frequency content to reduce the overall number of queries



SimBA Attack

5.4.3 Simple Black-box Attack

- Steps:
 - Randomly sample perturbation vectors from a predefined orthonormal basis
 - Check whether the perturbation increases misclassification probability
 - If successful, proceed; otherwise, try a different perturbation direction
 - Repeat until the model misclassifies the image
- Goal:
 - Each iteration moves the image away from the original image, and towards the decision boundary

SimBA Attack

5.4.3 Simple Black-box Attack

- Algorithm
 - Random director vectors $\mathbf{q}$ are sampled, and perturbation with step size ϵ are added or subtracted to misclassify the image

Algorithm 1 SimBA in Pseudocode

```
1: procedure SIMBA( $\mathbf{x}, y, Q, \epsilon$ )
2:    $\delta = 0$ 
3:    $\mathbf{p} = p_h(y \mid \mathbf{x})$ 
4:   while  $\mathbf{p}_y = \max_{y'} \mathbf{p}_{y'}$  do
5:     Pick randomly without replacement:  $\mathbf{q} \in Q$ 
6:     for  $\alpha \in \{\epsilon, -\epsilon\}$  do
7:        $\mathbf{p}' = p_h(y \mid \mathbf{x} + \delta + \alpha \mathbf{q})$ 
8:       if  $\mathbf{p}'_y < \mathbf{p}_y$  then
9:          $\delta = \delta + \alpha \mathbf{q}$ 
10:         $\mathbf{p} = \mathbf{p}'$ 
11:       break
   return  $\delta$ 
```

SimBA Attack

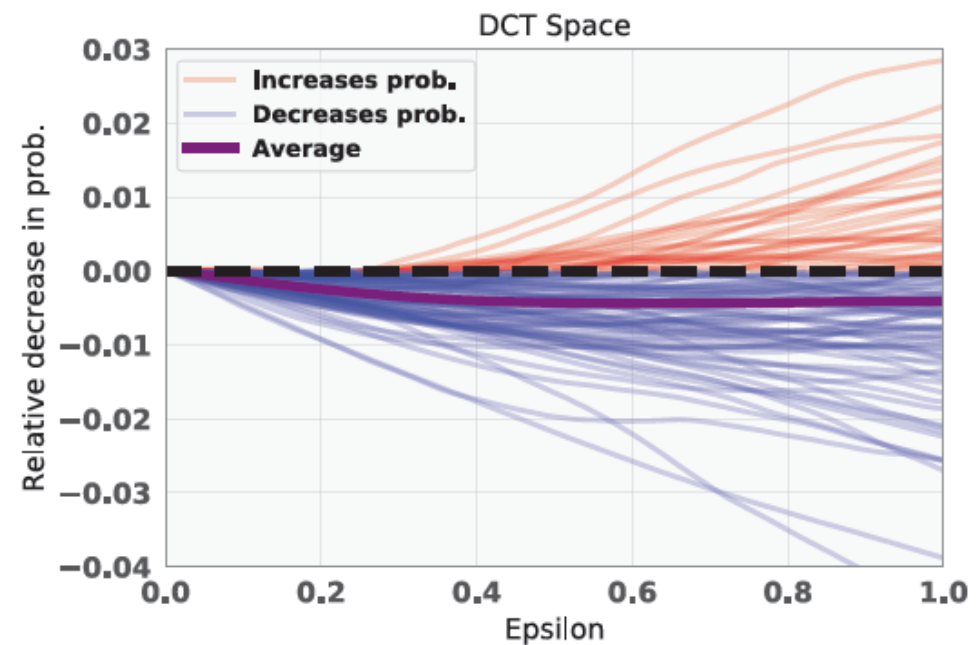
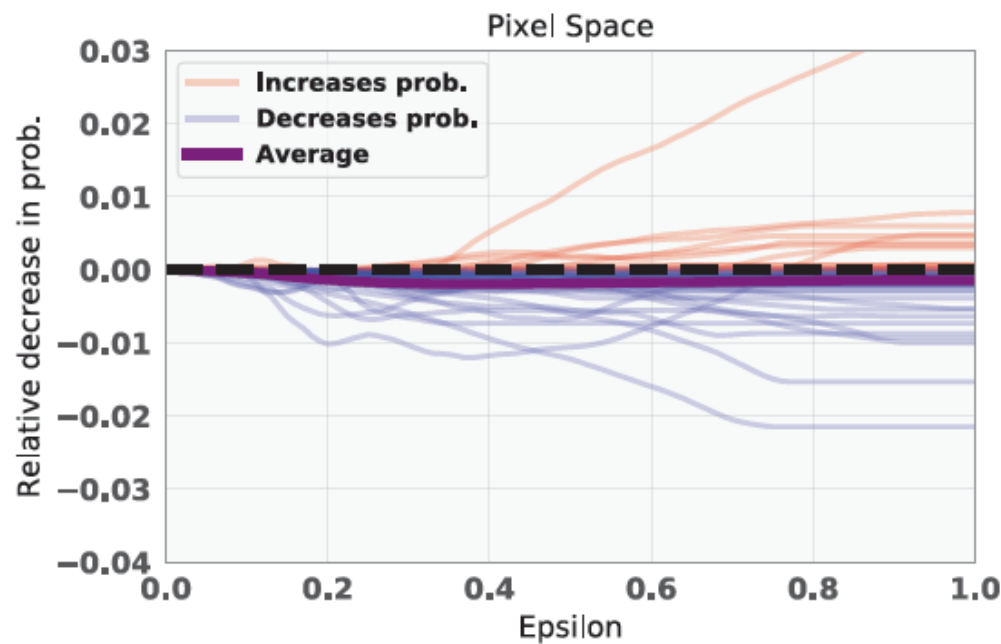
5.4.3 Simple Black-box Attack

- Perturbation vectors are selected to be orthonormal
 - I.e., the random directions for each pixel do not cancel each other out, or amplify each other
- For **orthonormal vectors** $\mathbf{x}$ and $\mathbf{y}$, their dot product is $\mathbf{x} \cdot \mathbf{y} = 0$
 - The angle between the vectors is 90 degrees
 - I.e., they are orthogonal
- How to choose orthonormal perturbation vector?
 - One inefficient option are the vectors $[1,0,0,\dots,0]$, $[0,1,0,\dots,0]$, $[0,0,1,\dots,0], \dots, [0,0,0,\dots,1]$
 - I.e., only one pixel is changed at a time
 - The authors propose an approach called **Discrete Cosine Transform (DCT)**
 - It is based on frequency coefficients that correspond to the magnitudes of cosine functions
 - This approach applies perturbations in the frequency domain, modifying specific frequency components of the image
 - I.e., low-frequency regions in images (e.g., image background) change less at each step, and therefore are queried less
 - The approach focuses on querying and applying perturbations to high-frequency regions in images

SimBA Attack

5.4.3 Simple Black-box Attack

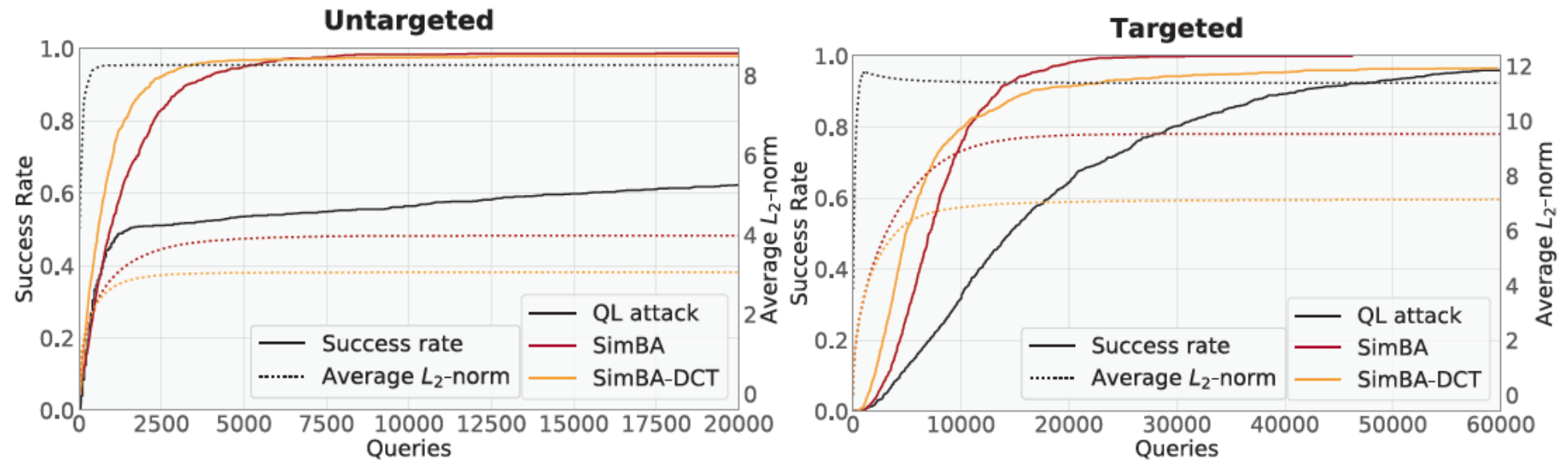
- The average change of the output probability scores is larger when the DCT approach is employed, in comparison to changing individual pixels
 - I.e., SimBA attack with DCT can find perturbations for many pixels in a single query that impact the output probability



SimBA Attack

5.4.3 Simple Black-box Attack

- Experimental evaluation
 - Full lines display attack success rate, dotted lines display average perturbation
 - SimBA attacks achieved high success rate with small average ℓ_2 norm, and fewer queries



SimBA Attack

5.4.3 Simple Black-box Attack

- Experimental evaluation
 - SimBA achieved good query-efficiency

Untargeted			
Attack	Average queries	Average L_2	Success rate
Label-only			
Boundary attack	123,407	5.98	100%
Opt-attack	71,100	6.98	100%
LFBA	30,000	6.34	100%
Score-based			
QL-attack	28,174	8.27	85.4%
Bandits-TD	5,251	5.00	80.5%
SimBA	1,665	3.98	98.6%
SimBA-DCT	1,283	3.06	97.8%

Targeted			
Attack	Average queries	Average L_2	Success rate
Score-based			
QL-attack	20,614	11.39	98.7%
AutoZOOM	13,525	26.74	100%
SimBA	7,899	9.53	100%
SimBA-DCT	8,824	7.04	96.5%

SimBA Attack

5.4.3 Simple Black-box Attack

- Attack on [Google Cloud Vision API](#)
 - Checked on 50 random images
 - 70% success rate after 5,000 queries



origin_54.BMP

Camera Accessory	87%
Product	82%
Hardware	67%
Optical Instrument	66%
Camera Lens	61%
Gun	61%
Product	58%
Weapon	53%



after_54.BMP

Weapon	94%
Gun	94%
Firearm	76%
Air Gun	65%
Trigger	63%
Optical Instrument	59%
Airsoft Gun	58%
Rifle	51%